LESSON 03 · CHEAT SHEET

Lambda Expressions in Depth

Java 8 → Java 21 · every error message below is real javac output

Key rules

1. **Shape:** `(params) -> expression` or `(params) -> { statements; return x; }` 8
2. **Parentheses** are optional only for one parameter with an inferred type.
3. **Expression body** returns its value automatically; a **block body** needs `return`.
4. **Target typing:** a lambda has no type of its own; the context (assignment, argument, return, `?:`, cast) decides.
5. **Capture:** locals must be final or **effectively final**. The lambda keeps a copy of the value. Fields are read live.
6. **this** = the enclosing instance. A lambda adds no new scope, so a parameter cannot reuse a local's name.
7. **var** in parameters 11: all or none; use it only for `final` or annotations.
8. **Checked exceptions:** java.util.function methods declare no `throws`.

Real compiler errors

Code	javac says
`ids.forEach(id -> count++);`	local variables referenced from a lambda expression must be final or effectively final
`String id; … id -> id.trim()`	variable id is already defined in method
`(var a, b) -> a + b`	invalid lambda parameter declaration (cannot mix 'var' and implicitly-typed parameters)
`Object o = () -> "x";`	incompatible types: Object is not a functional interface
`map(s -> Amounts.parse(s))`	unreported exception InvalidAmountException; must be caught or declared to be thrown
`(a, _) -> a` on 21	unnamed variables are a preview feature (final in Java 22)

Lambda vs anonymous class

	Lambda	Anonymous class
this	enclosing instance	the anonymous object
.class file	none (invokedynamic)	Outer$1.class
run time	hidden class 15	normal class
state / 2+ methods	no	yes

Syntax forms

Form	Example	Result
no params	`() -> "statement ready"`	statement ready
one, inferred	`t -> t.id()`	T003
explicit type	`(Transaction t) -> t.id()`	T003
two params	`(a, b) -> a.add(b)`	165.50
block body	`a -> { …; return a.add(fee); }`	2512.50
var 11	`(final var t) -> t.id()`	T010

Canonical snippets

1 · Capture: effectively final, loops

```
BigDecimal rate = percent(pct); // set once
return a -> a.multiply(rate).setScale(2, HALF_UP);
for (Flight f : flights) list.add(() -> f.code());
for (int i = 0; …; i++) { int pos = i + 1; … }
```

2 · Count without a captured counter

```
// smell: AtomicInteger / int[] inside forEach
long debits = txns.stream()
    .filter(t -> t.type() == DEBIT)
    .count();                          // 45
```

3 · Checked exceptions: wrap once, keep the cause

```
@FunctionalInterface
interface ThrowingFunction<T,R,E extends Exception> {
    R apply(T t) throws E;
}
raw.stream()
   .map(unchecked(Amounts::parse))  // cause kept
   .toList();
```

4 patterns for checked exceptions

1. try/catch inside the lambda (verbose)
2. custom SAM that declares `throws`; caller handles it
3. `unchecked(…)` wrapper, unwrap at the edge
4. failure as a value: `Optional` now, Result in L17

Do / Don't

✔ Do

- One expression; > 3 lines → named method
- No side effects in map / filter

✘ Don't

- Counter hacks
- Empty catch to make it compile
- == or lock on a lambda

Lesson 03 · sticky notes

Cut out, stick on your wall, and read one every morning.

(params) -> body

Expression: value returned.
Block { }: needs return.
( ) optional for 1 inferred param.

Target typing

Same lambda text →
Predicate, Function, or my own type.
The context decides.

Effectively final

Captured locals are never re-assigned.
The lambda keeps a COPY.

No counter hacks

AtomicInteger / int[] in forEach = smell.
`filter(...).count()` → 45

this = outer object

Lambda: enclosing instance.
Anonymous class: itself.
No new scope.

var [11]: all or none

`(final var t) -> ...`
Only for final or annotations.
`_` = Java 22

Checked exceptions

Custom SAM with throws,
or wrap once & keep the cause.

> 3 lines?

Give it a name.
Extract a method, use a method reference (L04).